Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

Examples: Using the API to Retrieve Connector Details

You can use the Stellar Cyber API to query the DP for a list of its configured connectors, either for a specified tenant or, if you have the necessary privileges, for all tenants. The API returns detailed configuration information on all connectors matching the query.

Refer to **Configuring API Authentication** for general requirements to use the API.

API Syntax for Retrieving Connector Details

The syntax for retrieving connector details via the API is as follows:

- Query for all connectors:

  `https://URL/connect/api/v1/connector`

- Query for connectors for a specified tenant:

  `https://URL/connect/api/v1/connector?cust_id=<tenant_id>`

Arguments:

- **<tenant_id>** – Optional. You can use this argument to query for all connectors assigned to a specified tenant using its ID retrieved from **System | ORGANIZATION MANAGEMENT | Tenants**, as described below.

In response, the API returns detailed information on all connectors matching the API call. The information returned is similar to that presented in the System | INTEGRATIONS | Connectors page and is explained below in **Connector Information Returned by the API**.

Finding the Tenant ID

To query for the connectors belonging to a specific Tenant, you must know its numerical Tenant ID. You can find this value as follows:

1. Navigate to **System | INTEGRATIONS | Connectors**and sort it by the **Tenant Name** column.
2. Note the name of the Tenant for whose connectors you want to query.

3. Navigate to **System | ORGANIZATION MANAGEMENT | Tenants** and sort it by **Tenant Name**.
4. Locate the table entry for the Tenant for which you want to query.
5. Right-click in the **ID** column and use the **Copy to Clipboard** command in the context menu that appears to copy the Tenant ID.
6. Paste the Tenant ID into a text file to store it temporarily.

Examples

The following sections provide examples of common ways to use the connector API. The examples all use Python, though any programming language will work.

Querying for All Connectors Across All Tenants

The following example uses a Python script to query for all connectors across all tenants with the following details:

- **Stellar Cyber DP IP Address** – myserver.stellarcyber.cloud
- **Username** – myuser@stellarcyber.ai
- **API Key (Refresh Token) from GUI** – 2iRpBAyQYEfv77R2QtATlJN6Nvq6uzftBdzotSy2pjT-IvJTLw9aiHyh7Y2mo12IDSWc-FfHwUyPpmiHQnJrSH


Because this script queries for all connectors across all tenants, the Tenant ID is included.

Understanding the Script

This script works as follows:

- The script sets the **host** and **refresh_token** parameters in **Step 1** in the sample.
- Because JWTs expire ten minutes after they are generated, this script includes logic that generates and uses a fresh JWT every time the script is run. The script runs the **getAccessToken** procedure to generate the new JWT (**Step 2** in the sample).
- The script uses the generated JWT to make a call to the connector API in the **getConnectors** procedure (**Step 3** in the sample).
- The script also prints the generated JWT to the screen. This, however, is not strictly necessary since the **getAccessToken** procedure already prints the status code for the call to the **access_token** API (**200** for success; **401** for failure).

```
#!/usr/bin/python3

import requests
import base64
import json
from urllib.parse import urlunparse
requests.packages.urllib3.disable_warnings()

# Step 1
# Add DP IP/hostname and refresh token from GUI here
HOST = "myserver.stellarcyber.cloud"
refresh_token =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZX
JfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYj
QyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2Senlw
UTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo"
```

```
def getAccessToken(refresh_token):
    headers = {
        "Authorization": "Bearer " + refresh_token
    }
    url = urlunparse(("https", HOST, "/connect/api/v1/access_token", "", "", ""))
    res = requests.post(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()["access_token"]

def getConnectors(token):
    headers = {"Authorization": "Bearer " + token}
    url = urlunparse(("https", HOST, "/connect/api/v1/connector", "", "", ""))
    res = requests.get(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()

if __name__ == "__main__":

    # Step 2: Use getAccessToken with supplied credentials to generate JWT
    jwt = getAccessToken(refresh_token)
    print("------------ jwt -------------")
    print(jwt)
    print("------------ jwt  end -------------")

    # Step 3: use JWT token to call public API
    connectors = getConnectors(jwt)
    print("------------ call result of /connect/api/v1/connector -------------")
    print(connectors)
    print("------------ end api results -------------")
```

Querying for a Single Tenant's Connectors

The following example uses a Python script to query for all connectors belonging to a specific tenant with the following details:

- **Stellar Cyber DP IP Address** – myserver.stellarcyber.cloud
- **API Key (Refresh Token) from GUI** – eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZXJfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYjQyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2SenlwUTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo
- **Tenant ID (cust_id)** – 7de229c6998041d1b84d9cb0e8893752

Understanding the Script

This script works as follows:

- The script sets the **host** and **refresh_token** parameters in **Step 1** in the sample.
- Because JWTs expire ten minutes after they are generated, this script includes logic that generates and uses a fresh JWT every time the script is run. The script runs the **getAccessToken** procedure to generate the new JWT (**Step 2** in the sample).
- The script uses the generated JWT to make a call to the connector API in the **getConnectors** procedure (**Step 3** in the sample). The script includes the Tenant_ID in the URL.
- The script also prints the generated JWT to the screen. This, however, is not strictly necessary since the **getAccessToken** procedure already prints the status code for the call to the **access_token** API (**200** for success; **401** for failure).

```
#!/usr/bin/python3

import requests
```

```
import base64
import json
from urllib.parse import urlunparse
requests.packages.urllib3.disable_warnings()

# Step 1
# Add DP IP/hostname and refresh token from GUI here
HOST = "myserver.stellarcyber.cloud"
refresh_token =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZX
JfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYj
QyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2Senlw
UTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo"

def getAccessToken(refresh_token):
    headers = {
        "Authorization": "Bearer " + refresh_token
    }
    url = urlunparse(("https", HOST, "/connect/api/v1/access_token", "", "", ""))
    res = requests.post(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()["access_token"]

def getSensors(token):
    headers = {"Authorization": "Bearer " + token}
    url = urlunparse(("https", HOST, "/connect/api/v1/connector?
cust_id=7de229c6998041d1b84d9cb0e8893752", "", "", ""))
    res = requests.get(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()

if __name__ == "__main__":

    # Step 2: Use getAccessToken with supplied credentials to generate JWT
    jwt = getAccessToken(refresh_token)
    print("------------ jwt -------------")
    print(jwt)
    print("------------ jwt  end -------------")

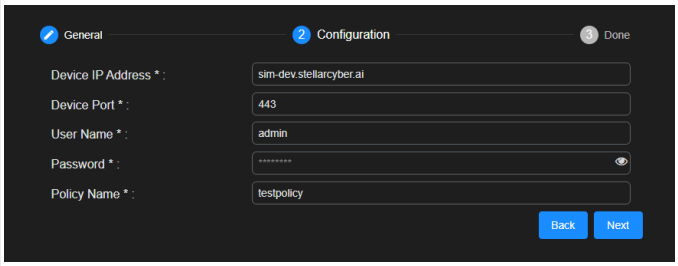
    # Step 3: use JWT token to call public API
    sensors = getSensors(jwt)
    print("------------ call result of /connect/api/v1/sensors -------------")
    print(sensors)
    print("------------ end api results -------------")
```

Connector Information Returned by the API

The API returns the following information for each connector matching the API call in field:value pairs, with fields separated from values by a colon. Separate field:value pairs are separated with commas for easy import. If a field:value pair has subvalues, they are listed inside pairs of backward slashes (for example, **\"domain\":\"SMC User\"**); refer to **Sample Output** for an example.

These fields are all specified when the connector is added to Stellar Cyber.

API Field Name	Description
total	The total number of connector entries returned for the query, followed by the type of entity returned for the query (**connectors** in this context). For example: `{"total":88,"connectors"`
_id	The internal ID for the connector.
category	The category assigned to the connector when it was created. For example, **Cloud Security**, **Firewall**, and so on.
configuration	The configuration section provides different details depending on the connector's category. The information reported corresponds directly to **Step 2 Configuration** in the Add Connector/Edit Connector workflow.

API Field Name	Description
	For example, the **Configuration** step for a **Firewall** connector looks like this:  The configuration API output for this same connector appears like this: {"category":"firewall" "configuration": "{\"username\":\"admin\" \"domain\":\"SMC User\" \"aella_tenantid\":\"\" \"dst_ip_grp_name\":\"StellarCyber_DstGrp\" \"policy_name\":\"standard\" \"src_ip_grp_name\":\"StellarCyber_SrcGrp\" \"device\":\" 71.211\" \"log_type\":\"checkpoint_firewall\"}" Refer to **Working with the Connectors Table** for details on the different fields available for each connector type.
tenant_id	The numerical Tenant ID for the connector. Tenant IDs are listed in System | ORGANIZATION MANAGEMENT | Tenants.
is_collect	Indicates whether the **Collect** funtion is enabled for the connector.
is_respond	Indicates whether the **Respond** function is enabled for the connector.
name	The name assigned to the connector.
run_on	Indicates whether the connector runs on the DP (**dp**) or a data sensor (**ds**).
type	Shows the connector type (for example, Checkpoint, MySQL, and so on).
version	The version number for the connector.
last_activity	The last activity time for the connector, expressed in epoch time.
last_data_received	The last time data was received by this connector, expressed in epoch time.
status	The health status of the connector.
active	Indicates whether the connector is active or not (boolean).
filter_list	The log filters applied to this connector.
created_at	The time at which the connector was first created, expressed in epoch time.
modified_at	The last time at which the connector was modified, expressed in epoch time.

Sample Output

The text below shows sample output of an API call for a specific tenant's connectors. In this case, the tenant has only one connector assigned (a database connector):

{'total': 1, 'connectors': [{'_id': '6542fa41cc22384bd8b468b7', 'active': True, 'category': 'endpoint', 'configuration': '{"interval":5,"oid":"7138a1e2-ebb6-44d8-9666-56a7ca7d48c4","aella_tenantid":"d7e9259bf0cb43578ad6bdae2feb7d99","content_type":["alerts"],"host_url":"https://api.limacharlie.io/v1","log_type":"limacharlie"}', 'created_at': 1698888257, 'tenantid': 'd7e9259bf0cb43578ad6bdae2feb7d99', 'filter_list': [], 'is_collect': True, 'is_respond': False, 'last_activity': 1702375700274, 'last_data_received': 1702341772793, 'name': 'Limacharlie-alert', 'run_on': 'dp', 'status': {'code': 0, 'message': None, 'status_time': 1702375599789}, 'type': 'limacharlie', 'version': '03.11.0000'}]}

© 2026 Stellar Cyber . All rights reserved.